Securing Open-Source Software via Fuzz Testing

Keegan Ravindran
Singapore Management University
keeganr.2023@scis.smu.edu.sg

Abstract

Open-source software (OSS) underpins modern computing, yet its security is hindered by inconsistent testing, limited maintainer resources, and inadequate vulnerability triaging. This paper reviews existing OSS security evaluation methods, including OpenSSF Scorecard, Bus Factor, and maintainer responsiveness metrics, identifying gaps such as low fuzzing adoption, delayed CVE handling, and insufficient project-specific nuance. To address these, we analyze a dataset of pre-installed Ubuntu 24.04 LTS packages, apply Scorecard evaluations, and select low-scoring projects for targeted fuzz-testing with AFL++. We document challenges in harness development, propose metric refinements, and demonstrate how fuzzing can expose overlooked attack surfaces. Our work offers a practical framework that complements automated assessments with context-aware scoring to better prioritize OSS security improvements.

1 Introduction

Open-source software (OSS) forms the backbone of modern computing, including critical applications pre-installed on popular Linux distributions. Wermke et al. (2022) report that 95% of IT organizations rely on OSS for infrastructure. However, OSS security is often underestimated due to limited maintainer resources and the absence of thorough assessments. Many projects lack rigorous testing, leaving them exposed to exploits. Smaller OSS projects, in particular, often rely on informal trust, lack incident response procedures, and struggle with contributor vetting. Wermke et al. (2022) A single vulnerability—such as a Use-After-Free (UAF) error, Out-of-Bounds (OOB) access, or other memory corruption bug—can lead to severe system failures. This risk is compounded by a 741% rise in supply chain attacks in recent years. Sonatype (2022) As many OSS

components serve foundational roles in operating systems, a single exploit could jeopardize an entire system.

Fuzz testing has emerged as an effective method for uncovering security bugs, prompting initiatives like Google's OSS-Fuzz to encourage its adoption in open-source workflows. Nevertheless, many widely used components and pre-installed Linux utilities remain under-tested.

Several factors contribute to this gap:

- **Limited resources:** Small teams often lack the capacity for dedicated security testing.
- **Security deprioritization:** Some projects prioritize performance and functionality over security.
- **Fuzzing complexity:** Building effective fuzzing harnesses for utilities can be technically challenging.

This research seeks to bridge this gap by investigating the security posture of widely used OSS projects and applying fuzz testing to uncover vulnerabilities. We first collect a dataset of OSS components pre-installed in Linux distributions or widely adopted in critical environments. Using existing security evaluation tools, we establish a baseline to identify projects with potentially weaker security practices. Selected targets are then re-evaluated through fuzzing to improve their security posture.

Beyond identifying vulnerabilities, this research measures fuzzing coverage improvements in under-tested software. By integrating fuzz testing and responsible disclosure, we aim to contribute to the security reinforcement of critical OSS ecosystems.

1.1 Research Questions and Goals

To guide this research, we define the following research questions:

- **RQ1:** What is the current state of security assessment for Open-Source Software, and what are some possible improvements for such evaluation mechanisms?
- **RQ2:** Can fuzz testing help in securing these software and unveiling bugs or vulnerabilities within these projects?

To address these questions, our research is structured around three key goals:

1. Conduct a literature review to examine current security evaluation approaches and metrics for OSS. Based on these findings, we propose a refined security scoring framework.
2. Assess the security posture of pre-installed OSS projects in Linux systems and construct a representative dataset of under-tested and security-critical packages.
3. Apply fuzz-testing using tools such as AFL++ to uncover vulnerabilities and evaluate the effectiveness of fuzzing in improving security for these OSS components.

2 Literature Review: Evaluating the Security of Open-Source Software

As the adoption of OSS becomes increasingly widespread, the challenge of ensuring OSS security intensifies. This section surveys the current state of OSS security evaluation, explores existing metrics and their limitations, and highlights emerging directions for more comprehensive assessment frameworks.

Focusing on general-purpose security assessment methods, we synthesize recent academic findings (within the past five years) and major industry practices to answer our first research question.

2.1 Motivation and Background

Several high-profile security incidents have underscored the importance of evaluating and securing OSS. Vulnerabilities such as Heartbleed (OpenSSL), Log4Shell (Log4j), and recent remote code execution (RCE) flaws in CUPS (Common Unix Printing System) highlight the systemic risk posed by flaws in widely-deployed OSS components. These vulnerabilities often propagate across the software supply chain, affecting downstream users and systems.

As noted by Ayala et al. (2025), the OSS ecosystem is burdened by a large and growing backlog of untriaged vulnerabilities, with over 197,000 entries in GitHub's security advisory database as of 2023. Many OSS projects lack formal disclosure or triaging processes, resulting in delays that leave critical flaws unaddressed. This emphasizes the need for robust evaluation metrics that go beyond superficial indicators.

2.2 Current State of OSS Security Evaluation

Contemporary OSS security evaluations incorporate multiple layers of measurement, each contributing a partial view of a project's security posture:

- **Known Vulnerability Scanning:** Tools like Snyk and Dependabot rely on public vulnerability databases (e.g., CVEs) to detect insecure dependencies. However, studies (Ayala et al., 2025) have shown that vulnerability disclosures often go unreviewed or unpatched, leaving significant blind spots.
- **Project Maintenance Metrics:** Indicators such as update frequency, contributor diversity, and the "bus factor" are used as proxies for project sustainability. (Nourry et al., 2024) found that 89% of 36,000 OSS projects had lost their core developers at some point, signaling potential security neglect.
- **Security Practice Frameworks:** Projects can self-attest to best practices via badges (e.g., OpenSSF Best Practices Badge) or standards like NIST SSDF. However, these frameworks are rarely enforced or independently verified.
- **Automated Security Scorecards:** OpenSSF's Scorecard evaluates projects using over a dozen metrics—including use of CI, fuzzing, signed releases, and dependency pinning—to generate a composite score (Zahan et al., 2023). While valuable, its scope is limited to platforms like GitHub and cannot assess all repositories uniformly.
- **Security Response Transparency:** The maturity of a project's vulnerability disclosure process is another indicator. Ayala et al. (2025) noted a backlog of over 63,000 vulnerabilities in unreviewed GitHub advisories.
- **Static and Dynamic Analysis Tools:** The use of fuzz testing (e.g., OSS-Fuzz enrollment) and static analyzers like CodeQL is considered a strong positive signal. Yet, Zahan et al. (2023) report that very few npm and PyPI packages utilize fuzzing.

2.3 Key Security Evaluation Metrics in Practice

To better understand how open-source software (OSS) security is assessed in practice, this section highlights three widely discussed evaluation metrics: the OpenSSF Scorecard, the Bus/Truck Factor, and Maintainer Responsiveness (also known as Time-to-Patch). These metrics are not only

academically validated but also play an increasingly prominent role in automated and manual assessments of OSS projects.

2.3.1 OpenSSF Scorecard

The OpenSSF Scorecard is a widely adopted tool that performs automated security checks on GitHub repositories across 18 dimensions, including code review, fuzzing, use of continuous integration, branch protection, signed releases, and more. Each check is scored from 0 to 10, and a weighted aggregate score is produced based on the criticality of each check.

Zahan et al. (2023) conducted a large-scale empirical evaluation of the Scorecard across thousands of packages in the npm and PyPI ecosystems. Their study revealed widespread failures in key checks: only 27% of packages had enforced code review, and the average fuzzing score across both ecosystems was 0.0, indicating that fuzzing remains critically underutilized.

While the Scorecard offers a scalable and standardized method of evaluation, it has several limitations:

- It currently supports only GitHub-hosted repositories fully.
- Certain checks are surface-level and can be gamed (e.g., dummy CI workflows).
- Fuzzing is treated as a binary indicator based on the presence of any harness, with no regard for its depth or effectiveness.

Nevertheless, its ease of use and automation make it a powerful baseline tool, and it is widely used in both academia and industry.

2.3.2 Bus/Truck Factor

The Bus Factor, also known as the Truck Factor, quantifies the risk associated with developer attrition in a project. Specifically, it measures the number of contributors whose loss would incapacitate ongoing development or maintenance.

Nourry et al. (2024) analyzed over 36,000 OSS projects and found that 89% had lost their core developers at some point, with 70% experiencing this within the first three years. Alarmingly, only 27% of projects were able to successfully replace their core maintainers.

The Bus Factor is a useful metric for evaluating:

- Project sustainability and long-term viability.
- Risk of stagnation, unpatched vulnerabilities, and delayed security responses.

However, it does not directly measure technical security quality, test coverage, or responsiveness to issues. Therefore, it is best used in conjunction with other metrics.

2.3.3 Time-to-Patch and Maintainer Responsiveness

Time-to-Patch refers to the duration between the discovery/reporting of a vulnerability and its resolution. Maintainer responsiveness includes how quickly maintainers acknowledge and address issues, pull requests, and security advisories.

Ayala et al. (2025) highlighted a significant backlog in GitHub's Security Advisory database, with over 197,000 unreviewed advisories. Many of these advisories included confirmed vulnerabilities that remained untriaged and unpatched. Such delays are especially prevalent in projects with limited contributor bases or informal triaging processes.

Although it can be influenced by issue severity or project size, Time-to-Patch is a strong real-world indicator of a project's ability to manage and mitigate risk in a timely manner. Projects with quick patch cycles tend to expose users to less risk from known vulnerabilities.

Together, these three metrics — OpenSSF Scorecard, Bus/Truck Factor, and Maintainer Responsiveness—offer complementary insights into the health and security posture of OSS projects. By incorporating them into evaluation frameworks, researchers and practitioners can better identify at-risk components and prioritize intervention efforts.

2.4 Limitations in Current Approaches

Despite recent progress, several gaps hinder the reliability and scope of existing evaluation mechanisms:

- **Narrow Scope of Metrics:** Evaluations tend to focus on known vulnerabilities, missing risks such as inactive maintainers or outdated dependencies(OWASP Foundation, 2023).

- **Inconsistent Standards:** Security checklists vary across frameworks, making it difficult to enforce a universal baseline (Open Source Security Foundation, 2022).
- **Tool Limitations:** Tools like Scorecard cannot assess repositories outside mainstream forges (e.g., GitHub, GitLab), leading to incomplete coverage.
- **Backlog of Vulnerabilities:** Ayala et al. (2025) emphasize that untriaged disclosures undermine the effectiveness of automated evaluations.
- **Lack of Testing Adoption:** Fuzzing and SAST are not widely adopted among smaller OSS projects, which tend to lack the resources to implement these measures.

2.5 Proposed Improvements from the Literature

To address these challenges, recent research and community efforts have suggested several improvements:

- **Standardizing Metrics:** Zahan et al. (2023) advocate for a unified, consensus-based framework of measurable security practices.
- **Expanding Tool Coverage:** Improving Scorecard's support for non-GitHub platforms and integrating additional data sources like OSV could broaden its reach.
- **Holistic Risk Models:** OWASP Foundation (2023) proposes incorporating broader risk dimensions such as maintainer activity, package popularity, and supply chain risks.
- **Enhancing Vulnerability Disclosure:** Streamlining CVE assignment and integrating bug bounty data can close gaps in vulnerability awareness (Ayala et al., 2025).
- **Promoting Proactive Testing:** Enabling easier fuzzing and SAST integration—via CI pipelines or grant-based initiatives—can improve adoption among smaller OSS projects.
- **Incentivizing Maintainers:** Synopsys (2023) and Snyk (2023) suggest providing funding and support for security fixes to reduce alert fatigue and promote responsiveness.

2.6 Conclusion

The security evaluation of OSS has matured significantly, with tools like OpenSSF Scorecard providing quantifiable, evidence-based assessments. However, a number of challenges remain, including inconsistent metrics, limited tool coverage, and insufficient testing adoption. The literature calls for holistic, proactive, and community-supported evaluation models that better align with the evolving OSS landscape.

2.7 Implications for this Research

This review reveals both the value and shortcomings of existing security evaluation approaches. While tools like Scorecard and OSS-Fuzz have improved observability and hygiene, they fall short of capturing the nuanced and evolving nature of OSS risk. There is a clear need for more holistic evaluation frameworks—potentially combining historical CVE behavior, community dynamics, fuzz testing results, and process maturity.

3 Methodology

3.1 Dataset Construction

The first phase of our research involved constructing a representative dataset of open-source software (OSS) projects for security evaluation. To maintain real-world practicality, we specifically targeted packages that are pre-installed on modern Linux operating systems, as these components are widely deployed but often receive limited individual security scrutiny and attention.

3.1.1 Choice of Ubuntu 24.04.2 LTS Minimal Installation

Ubuntu 24.04.2 LTS was selected as the base platform for several reasons. First, Ubuntu is among the most widely adopted Linux distributions, both for desktop and server environments, and benefits from an active upstream maintenance team providing five years of security updates for each Long Term Support (LTS) release. According to a W3Techs (2020) survey, Ubuntu accounts for approximately 14.7% of all websites running on Linux servers.

Second, using a minimal installation of Ubuntu ensures that the dataset captures the essential software stack without unnecessary bloat. Minimal installations deploy only the fundamental system components required for operation, excluding optional user-facing applications and non-critical utilities. This allows the study to focus on lower-level, often-overlooked packages that form the foundation of Linux systems, such as compression libraries, hardware listers, and networking tools.

Furthermore, pre-installed packages in minimal distributions are typically trusted implicitly by users and system administrators. Despite this trust, they may suffer from security neglect due to limited maintainer resources, less visibility compared to flagship applications, or historical legacy codebases. Thus, securing these foundational components is crucial to improving overall system resilience.

3.1.2 Enumerating and Filtering Packages

We conducted the dataset enumeration using the command:

```
apt list --installed
```

executed on a clean minimal installation of Ubuntu Server 24.04.2 LTS and Ubuntu Desktop 24.04.2 LTS. The Server image yielded 491 core packages while the Desktop image yielded 1336 packages. To narrow the scope of our research project, we focused on the packages from the Server image, which were also present in the Desktop installation. Each package was then manually investigated to locate its upstream source code repository.

For consistent automated evaluation, only packages whose source code was publicly hosted on GitHub were retained. This filtering allowed us to deploy OpenSSF's Scorecard as a baseline measurement, which is limited to Github-hosted repositories for full functionality. Projects hosted on other platforms such as Launchpad, SourceForge, Savannah, or custom domain-specific repositories were excluded from the dataset.

Following the filtering process, the final dataset consisted of 154 OSS repositories. Each repository was publicly accessible and eligible for standardized Scorecard evaluation.

3.1.3 Rationale for GitHub-only Filtering

While restricting the dataset to GitHub repositories introduces a certain selection bias, it ensures the consistency and comparability of automated security assessments. GitHub is currently the most popular OSS hosting platform, providing standard interfaces (e.g., GitHub Actions CI, Dependabot alerts) that are integrated into security evaluation tools such as Scorecard.

Repositories on alternative hosting platforms often lack the metadata or integration points necessary for Scorecard's automated checks. Therefore, GitHub-only filtering was a pragmatic decision to maintain evaluation quality, even though it necessarily excluded certain older or niche projects.

3.1.4 Baseline Security Evaluation

Once the dataset was finalized, we applied the OpenSSF Scorecard tool to each repository to generate a baseline security score. OpenSSF Scorecard is an automated framework that evaluates OSS projects against a set of best-practice security indicators, aiming to identify common weaknesses across repositories.

The Scorecard evaluates repositories across 18 security checks summarized in Table 1.

Table 1: OpenSSF Scorecard Security Checks

Check	**Description**
Branch-Protection	Ensures important branches are protected from force-pushes and require pull request reviews.
Code-Review	Verifies that all commits are reviewed by someone other than the original author.
Contributors	Checks if multiple organizations contribute to the project.
Dangerous-Workflow	Detects unsafe GitHub Actions workflows.
Dependency-Update-Tool	Verifies use of automated dependency update tools (e.g., Dependabot, Renovate).
Fuzzing	Checks for the use of fuzz testing (e.g., OSS-Fuzz).
License	Confirms the repository includes an OSS license file.
Maintained	Measures if the project is actively maintained based on recent commit activity.
Packaging	Evaluates secure software packaging and release practices.
Pinned-Dependencies	Checks if dependencies are pinned to specific versions.
SAST	Checks for use of Static Application Security Testing tools in CI pipelines.
Security-Policy	Verifies the presence of a documented security policy.
Signed-Releases	Verifies that project releases are cryptographically signed.
Token-Permissions	Ensures OAuth token permissions in GitHub Actions are minimized.
Vulnerabilities	Checks for unfixed vulnerabilities in dependencies.
Binary-Artifacts	Detects presence of prebuilt binary artifacts checked into the repository.
CI-Tests	Verifies that continuous integration testing is configured for contributions.
CII-Best-Practices	Checks if the project has achieved the CII Best Practices Badge.

Each metric is individually scored on a 0–10 scale based on the presence and robustness of the relevant practice. Metrics are also weighted differently based on the severity of the associated risk, with some checks categorized as *Critical*, *High*, *Medium*, or *Low* risk.

The overall Scorecard aggregate score is a weighted average of these checks, designed to reflect the security hygiene of a project. A higher score suggests stronger adherence to best practices and a lower likelihood of basic security weaknesses.

Applying Scorecard to our dataset of 154 GitHub repositories revealed several concerning trends:

- 57.79% of projects had a fuzzing score of zero, indicating no documented fuzz tests or OSS-Fuzz enrollment.

- The mean aggregate Scorecard score across all projects was 5.07, with a median of 5.10.
- The minimum aggregate score observed was 2.20, and the maximum was 9.50.
- The 25th and 75th percentile scores were 3.73 and 6.20, respectively.

These results highlight that even among pre-installed OSS packages in a widely used Linux distribution, significant gaps exist in adopting basic security practices such as dependency pinning, CI setup, and fuzzing. These observations informed the subsequent selection of target projects for further investigations.

3.1.5 Dataset Analysis

To better understand the security posture of the constructed dataset, we conducted a preliminary analysis based on the OpenSSF Scorecard results. This analysis provides insights into common security weaknesses and informed the subsequent selection of projects for fuzz-testing.

Figure 1 shows the distribution of fuzzing scores among the 154 evaluated OSS projects. Notably, 57.8% of the projects had a fuzzing score of zero, indicating no documented use of fuzz testing tools such as OSS-Fuzz. This suggests that dynamic analysis techniques remain significantly underutilized even among widely deployed Linux system components.

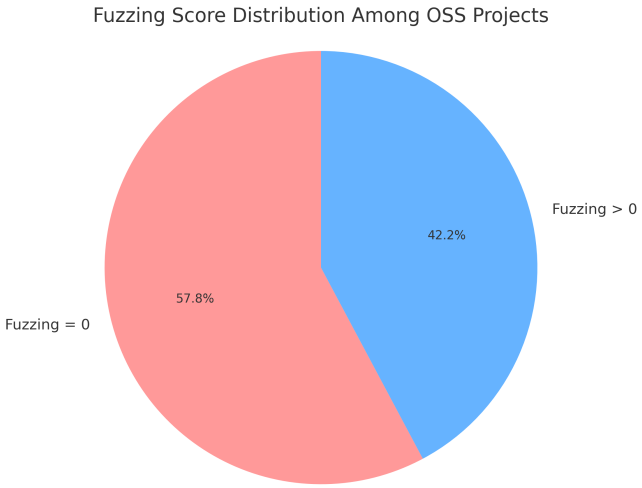

Figure 1: Fuzzing score distribution among OSS projects. 57.8% of projects had a fuzzing score of zero.

We also examined the distribution of aggregate Scorecard scores, presented in Figure 2. The mean aggregate score was 5.07, with a median of 5.10, indicating a moderate adherence to security best practices overall. However, the distribution is skewed, with a significant number of projects clustering between scores of 3.5 and 6.5. Only a minority of projects achieved scores above 8.0, demonstrating exceptional security hygiene.

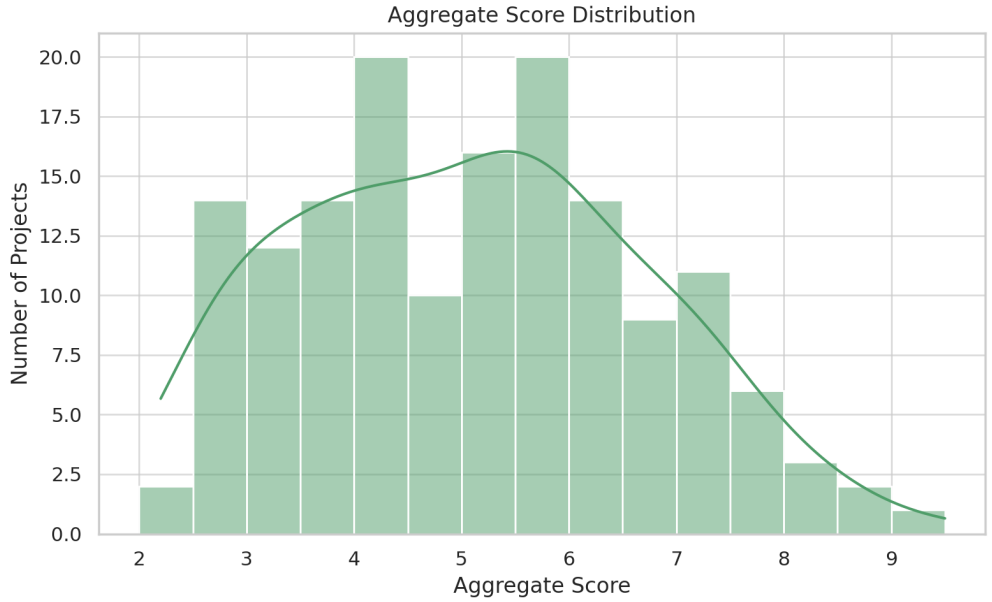

Figure 2: Distribution of OpenSSF Scorecard aggregate scores among 154 OSS projects.

In addition to aggregate scoring, we compared specific security metrics between all evaluated projects and the subset of projects lacking documented fuzz testing efforts. As shown in Figure 3, fuzzable projects consistently underperformed across key security dimensions, including token permissions, signed releases, packaging practices, and static analysis adoption (SAST). Notably, the average fuzzing score for these projects was near zero, reinforcing their suitability for deeper fuzz-testing interventions.

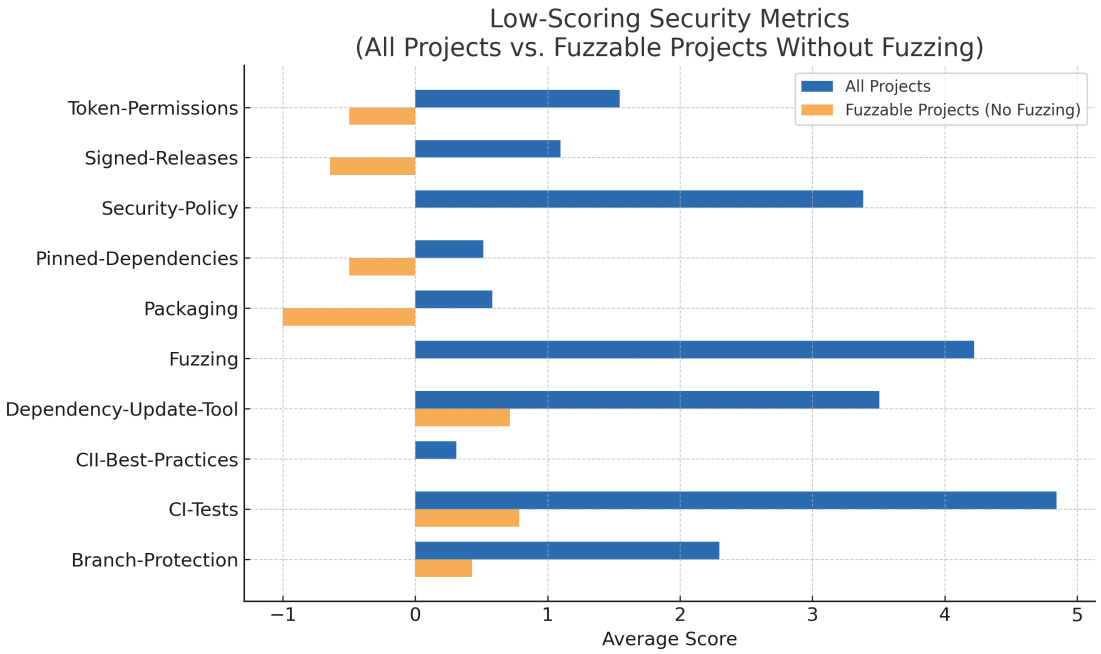

Figure 3: Comparison of low-scoring OpenSSF Scorecard security metrics between all projects and fuzzable projects without documented fuzz testing.

The dataset analysis highlights the existence of considerable variability in security practices among foundational OSS components. The observed trends—in particular, the low adoption of fuzz testing and inconsistent security policy enforcement—reinforce the need for targeted security interventions. These findings directly motivated the selection of specific low-scoring and under-tested projects for deeper manual fuzzing analysis in the subsequent phase of the study.

3.1.6 Selection of Targets for Deeper Analysis

From the dataset, we selected a subset of projects that exhibited low aggregate security scores and lack documented fuzzing efforts. The selected projects for deeper fuzz-testing analysis are summarized in Table 2.

Table 2: Selected Projects for Deeper Fuzz-Testing Analysis

Project	Description	Scorecard Aggregate
libmspack	A library for Microsoft compression formats (CAB, CHM, HLP, LIT, KWAJ, SZDD).	2.2
lshw	A hardware lister utility for Linux.	3.0
libusbmuxd6	A client library for handling usbmux protocol communications with iOS devices.	3.1
libisns0t64	An implementation of the Internet Storage Name Service (iSNS) protocol.	3.2
libimobiledevice6	A library enabling communication with iOS devices.	3.8

These selections represent a diverse cross-section of functionality—ranging from compression and hardware enumeration to mobile device communication—and were chosen to maximize impact and uncover different classes of vulnerabilities during subsequent fuzzing efforts.

3.1.7 Further investigation into Selected Targets

Following the initial selection of five OSS projects for deeper security analysis, we conducted a manual review of each project's Scorecard breakdown, security practices, and fuzzing potential. While Scorecard provides a useful high-level assessment, our investigation revealed several shortcomings in its evaluation, motivating the need for refined security metrics. This subsection summarizes our findings for each selected project.

libmspack

Description: A library for handling Microsoft compression formats (CAB, CHM, HLP, LIT, KWAJ, SZDD).

Table 3: Scorecard Results for libmspack

Metric	Score
Binary Artifacts	10.0
Branch Protection	0.0
CI Tests	-1
CII Best Practices	0.0
Code Review	0.0
Contributors	0
Dangerous Workflow	-1
Dependency Update Tool	0.0
Fuzzing	0.0
License	10.0
Maintained	0
Packaging	-1
Pinned Dependencies	-1
SAST	0.0
Security Policy	0.0
Signed Releases	-1
Token Permissions	-1
Vulnerabilities	10.0

Observations: While libmspack has a low aggregate score, we noticed that multiple metrics have a score of -1. This indicates that Scorecard was unable to determine any conclusive evidence of the presence/measure of the metric. For example, the Packaging and Signed-Releases metrics scored -1, which indicates that Scorecard was unable to decide if the repository had packaging or releases present at all. As these values are ignored in the aggregate scored calculation, it can lead to some skewing of the aggregate score, resulting in inaccurate reflections of the libraries' security posture. The developers of libmspack maintain a website to document the releases and packaging of the project, and only use GitHub as a version control system for collaborative development. Hence, this skewing caused by the negative values from signed-releases and packaging could have caused libmspack to have a lower aggregate score, misrepresenting its security posture.

libmspack also scored a 0 for security policy, suggesting that scorecard was unable to detect a security markdown file documenting the project's security policies. However, the libmspack homepage display's the maintainer's preferred method of informal communication for reporting bugs via e-mail, and tracks past CVEs. As such, even though the package does have some security policies

in place, albeit informal, Scorecard misrepresents it with a score of 0 as it was undetected on the GitHub repository, contributing to its lower aggregate score.

lshw

Description: A hardware lister utility for Linux systems that extracts detailed information about hardware configurations.

Table 4: Scorecard Results for lshw

Metric	Score
Binary Artifacts	10.0
Branch Protection	0.0
CI Tests	-1
CII Best Practices	0.0
Code Review	0.0
Contributors	10.0
Dangerous Workflow	-1
Dependency Update Tool	0.0
Fuzzing	0.0
License	10.0
Maintained	0.0
Packaging	-1
Pinned Dependencies	-1
SAST	0.0
Security Policy	0.0
Signed Releases	-1
Token Permissions	-1
Vulnerabilities	10.0

Observations: lshw has an aggregate score of 6, with 6 out of the 18 Scorecard metrics having invalid scores. lshw also scored a 0 for maintenance. However, considering that it is a small tool, updates may not be as frequent as more complex projects, and may not require frequent commits if the tool is already stable. Most of the core source code has not had any changes for many years, and not many new features are to be added. As such, the project might not have had many commits over the past 90 days, let alone 1 commit a week (which Scorecard uses for its Maintenance metric),

accounting for its low maintenance score.

libusbmuxd6

Description: A client library implementing the usbmux protocol for communication with iOS devices over USB.

Table 5: Scorecard Results for libusbmuxd6

Metric	Score
Binary Artifacts	10.0
Branch Protection	0.0
CI Tests	3.0
CII Best Practices	0.0
Code Review	1.0
Contributors	10.0
Dangerous Workflow	10.0
Dependency Update Tool	0.0
Fuzzing	0.0
License	10.0
Maintained	0.0
Packaging	-1
Pinned Dependencies	0.0
SAST	0.0
Security Policy	0.0
Signed Releases	0.0
Token Permissions	0.0
Vulnerabilities	10.0

Observations: libusbmuxd6 scorees a 0 for Code Review, Maintained and more metrics. Investigating the repository's GitHub page manually, we discovered that the latest issue raised on the GitHub Issue Tracker 3 weeks ago received discussion from the maintainers of the project within the same week. Does this project deserve a score of 0 for Maintenance due to lack of recent commits, despite the maintainers actively looking to better the project?

libisns0t64

Description: An implementation of the Internet Storage Name Service (iSNS) protocol.

Table 6: Scorecard Results for libisns0t64

Metric	Score
Binary Artifacts	10.0
Branch Protection	0.0
CI Tests	0.0
CII Best Practices	0.0
Code Review	3.0
Contributors	10.0
Dangerous Workflow	-1
Dependency Update Tool	0.0
Fuzzing	0.0
License	10.0
Maintained	0.0
Packaging	-1
Pinned Dependencies	-1
SAST	0.0
Security Policy	0.0
Signed Releases	-1
Token Permissions	-1
Vulnerabilities	10.0

Observations: libisns0t64, or open-isns, has 5 invalid scores, leaving the aggregate score to be calculated with the remaining 13 metrics, of which 7 scored 0. These include the CII-Best-Practices badge metric.

The CII Best Practices badge is a self-attested certification that follows a checklist of security-oriented best practices such as vulnerability reporting policies, automatic processes for rebuilding, SAST and more. However, this badge is voluntary and relies entirely on maintainer self-declaration. Many OSS projects may never apply for the badge. For example, maintainers of smaller or more niche projects such as open-isns may not be interested in going through the effort of obtaining the badge. Should these projects be penalised for not obtaining the badge, even though they might actually follow security-oriented best practices?

libimobiledevice6

Description: A library enabling communication with iOS devices through various protocols.

Table 7: Scorecard Results for libimobiledevice6

Metric	Score
Binary Artifacts	10.0
Branch Protection	0.0
CI Tests	-1
CII Best Practices	0.0
Code Review	0.0
Contributors	10.0
Dangerous Workflow	10.0
Dependency Update Tool	0.0
Fuzzing	0.0
License	10.0
Maintained	9.0
Packaging	-1
Pinned Dependencies	-1
SAST	0.0
Security Policy	0.0
Signed Releases	0.0
Token Permissions	0.0
Vulnerabilities	10.0

Observations: libimobiledevice6 is maintained by the same team behind libusbmuxd. Although a clear security policy is not defined in a markdown file present in the repository, security concerns and bugs are frequently raised and addressed within the GitHub Issue Tracker, and security patches are vetted through pull requests, as advised on the homepage. Although the security policy is not stated in a markdown file, the community of contributors around the project have utilised GitHub's systems to contribute to the security of the project.

3.1.8 Proposed Improvements for Metrics

With the aforementioned examples, it is clear that OpenSSF's Scorecard has its limitations. The metrics used in the OpenSSF Scorecard aggregate score calculation are rather surface-level, and projects that are security-aware might still score low on some metrics due to differing practices. This can result in low aggregate scores, which do not realistically reflect the security policy of these projects.

As such, we propose the following changes and additions to the OpenSSF Scorecard:

First, under the 'Maintained' check, Scorecard currently only checks the number of commits within the past 90 days. However, that does not paint the full picture of the activity of maintainers. A project could have frequent commits from a single developer, but still be fragile. So, I suggest including additional signals—like the number of active developers, how consistently contributions are made over time, and whether new contributors are joining. This would help approximate long-term sustainability, and take into account the Bus Factor as previously detailed.

Secondly, Scorecard's Fuzzing check only verifies whether a fuzzing harness is present and grades it with a binary score, a project either only scores 10 or 0—it does not assess the effectiveness of the harness. One way to improve this would be to incorporate code coverage data, such as function, branch, or line-level coverage. That would give us a much better indication of how thoroughly the code is being tested.

Beyond these, there are two other areas worth considering as a new metric. The first is whether a project has a clearly defined testing policy—such as documentation on testing completeness or the presence of a dedicated testing team. The second is to evaluate how well existing tests actually detect vulnerabilities—especially past CVEs. In other words, if we know a vulnerability was foumd, would the project's test suite have caught it? If a project has no testing at all present in its repository, it would not be able to detect any vulnerabilities, and would score the lowest score.

These refinements could make metrics in Scorecard more meaningful, especially when trying to prioritize security improvements over a large set of open-source projects, and also allow us to better evaluate the security posture of these projects.

3.1.9 Refining our rankings

To refine the scores for our preliminary dataset, we assessed the testing status of each of the repositories.

libmspack's test directory contains test cases for every past CVE, with a neatly organised test suite, with test data organized by filetype and 11 test files written in C. libmspack is given a score of 10 for our proposed new metric.

lshw, libuxbmuxd, and libimobiledevice do not contain any testing files within the GitHub Repository (in any branch). As such, there is no way to measure if it would detect any vulnerabilities, and are given scores of 0 for our proposed new metric.

open-isns contains 22 testing suits, each with multiple test cases. However, with no past CVEs, we are unable to assess if the test suite would catch any known security bugs. Investigating the commits of the repository did not expose any more information regarding this, as none of the recent commits or pull-requests were bug-fixes for notable vulnerabilties. This project scored a 5 for our new proposed metric.

As Scorecard's aggregate score calculation formula is not clearly defined in publicly available documentation, we weighed our new metric with a factor of 0.1 to match the magnitude of values of Scorecard's aggregate score, and added our new metric to the current aggregate to obtain new aggregate scores.

Table 8: Adjusted Aggregate Scores After Incorporating Testing-Status Metric

Package	**Old Aggregate**	**Testing-Status**	**New Aggregate**
libmspack	2.2	$(0.1)(10) = 1$	3.2
lshw	3.0	0	3.0
libusbmuxd6	3.1	0	3.1
libisns0t64	3.2	$(0.1)(5) = 0.5$	3.7
libimobiledevice6	3.8	0	3.8

With these new aggregate scores, libmspack's security posture is now ranked above that of lshw and libusbmuxd6, which do not have any unit tests present in their repositories. This could better represent the security posture of these projects.

3.2 Improving targeted projects' security with Fuzzing

Fuzzing has been proved to be an effective method in securing software and exposing vulnerabilities. As these projects within the preliminary dataset lack fuzzing efforts, we aim to improve the security posture of OSS Projects by incorporating fuzzing, beginning with the targets in our preliminary dataset.

In this study, we applied fuzz testing using AFL++ (American Fuzzy Lop Plus Plus), a modern greybox fuzzer that combines lightweight instrumentation with coverage-guided input generation. Each selected project was compiled with 'afl-clang-fast' and instrumented with AddressSanitizer (ASAN) to detect memory corruption errors such as buffer overflows and use-after-free vulnerabilities.

Our fuzzing campaigns aimed to measure two primary metrics: line coverage and crash count. Line coverage reflects how effectively the fuzzer exercised different parts of the codebase, while crash count indicates the number of unique program crashes discovered. Each fuzzing session was conducted over at least a continuous 24-hour period, and crashes were triaged for analysis.

3.2.1 Fuzzing libmspack

With our prior investigation, we discovered 21 past CVEs for libmspack, with the most recent filed in 2019, while the library has been continually updated over the past few years. This suggests that there might be a lack of testing in recent years for this library, or that its security might be overlooked. Going through past vulnerabilities, majority are caused by buffer overflows, from functions pertaining to reading compressed file formats. Keeping that in mind, we designed the following harnesses as shown in Appendix A.1. In order to optimise Fuzzing, we used PoCs from past CVEs of the CAB and KWAJ filetypes for the respective harnesses.

3.2.2 Fuzzing libusbmuxd

The next target chosen for fuzzing was libusbmuxd, which is a client library to handle usbmux protocols with iOS devices. Only one CVE has been filed for libusbmuxd, which was filed in 2012. This package was prioritized over the others within the preliminary dataset as it serves as a dependency for one other package in the dataset (libimobiledevice). We designed the following harnesses as shown in Appendix A.2

4 Results / Analysis

4.1 libmspack Fuzzing Results

With the first harness targeting CAB extraction functionality, we achieved a coverage of 6.84%, with 0 saved crashes and 0 saved hangs. This relatively low coverage can be attributed to the structural complexity of CAB archives, which often require well-formed headers and specific file layouts to exercise deeper parsing logic. Despite the limited coverage, the fuzzer was able to explore basic parsing paths and validate the robustness of the initial stages of CAB processing. The absence of crashes suggests that the tested code paths were relatively resilient under malformed input conditions, although it is possible that deeper vulnerabilities remain undiscovered due to the input constraints.

With the second harness targeting functionality related to the KWAJ filetype, we achieved a significantly higher coverage of 36.99%, accompanied by the discovery of 3 saved crashes and 2 saved hangs. The higher coverage suggests that the KWAJ format parser was either more permissive to malformed inputs or had a simpler internal structure, allowing the fuzzer to exercise a broader range of code paths. The discovery of crashes and hangs indicates the presence of potential vulnerabilities or input-handling weaknesses in the KWAJ processing logic. These results highlight the effectiveness of targeted fuzzing at uncovering bugs in less mature or less commonly tested code paths.

Overall, the fuzzing efforts demonstrated the variability in fuzzer effectiveness depending on the target file format and parser complexity. Future work could involve enhancing input mutation strategies, incorporating format-aware fuzzing, or augmenting the initial corpus with manually crafted seeds to further improve coverage and bug discovery rates across both file types.

4.2 libusbmuxd Fuzzing Results

With the harness targeting *plist_from_bin* to decode binary Apple Property List (plist) formats, we achieved a coverage of 6.56%. The low coverage could be attributed to two factors,

- Much of libusbmuxd's code is intended to operate in conjunction with a running usbmuxd daemon. In the absence of a live daemon during fuzzing, many logical paths relying on socket communication, device handling, and asynchronous event processing were effectively

unreachable.

- The primary input format targeted — binary plist — is highly structured and fragile. Minor mutations often caused early deserialization failures, preventing the fuzzer from reaching more complex processing logic involving parsed device lists, message types, and property extraction. This restricted the ability of the fuzzer to explore deeper branches within the codebase.

5 Conclusion

This project first assessed existing security evaluation metrics for open-source software projects, examining their ability to characterize project security posture. Through this evaluation, we identified potential weaknesses in the reliance on static indicators, noting that good development practices do not necessarily guarantee the absence of vulnerabilities. Based on these observations, we proposed improvements to existing evaluation frameworks to better account for dynamic analysis and practical testing efforts. To complement our analysis, we conducted fuzz testing using AFL++ on selected projects that lacked fuzzing coverage, aiming to strengthen their security posture. Through the construction of targeted fuzzing harnesses, we identified several crashes and hangs, although further triage is required to determine their security significance. These results highlight the limitations of static evaluation alone, and reinforce the value of integrating dynamic techniques such as fuzzing into broader security assessment approaches for open-source software. Future work could involve expanding fuzzing efforts, refining input generation strategies, and developing more comprehensive metrics that combine both static and dynamic security indicators.

A Fuzzing Harnesses

A.1 libmspack Harnesses

A.1.1 CAB Harness

```
1 #include <mspack.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
```

```c
int main(int argc, char **argv) {
    if (argc < 2)
        return 1;

    const char *input_file = argv[1];

    struct mscab_decompressor *cabd = mspack_create_cab_decompressor(NULL);
    if (!cabd)
        return 1;

    struct mscabd_cabinet *cab = cabd->open(cabd, input_file);
    if (!cab) {
        mspack_destroy_cab_decompressor(cabd);
        return 0;
    }


    struct mscabd_file *file;
    for (file = cab->files; file != NULL; file = file->next) {
        cabd->extract(cabd, cab, file);
    }

    volatile unsigned short set_id = cab->set_id;
    volatile unsigned short num_folders = 0;
    volatile unsigned short num_files = 0;

    struct mscabd_folder *folder;
    for (folder = cab->folders; folder != NULL; folder = folder->next) {
        num_folders++;
    }

    for (file = cab->files; file != NULL; file = file->next) {
        num_files++;
        volatile const char *filename = file->filename; // trigger filename
            ↪ parsing
    }

    cabd->close(cabd, cab);
    mspack_destroy_cab_decompressor(cabd);
    return 0;
}
```

Listing 1: CAB Fuzzing Harness for libmspack

A.1.2 KWAJ Harness

```c
#include <mspack.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main(int argc, char **argv) {
    if (argc < 2) {
        fprintf(stderr, "Usage: %s <kwaj_file>\n", argv[0]);
        return 1;
    }
    const char *input_file = argv[1];

    struct mskwaj_decompressor *kwajd = mspack_create_kwaj_decompressor(NULL);
    if (!kwajd) {
        fprintf(stderr, "Failed to create KWAJ decompressor\n");
        return 1;
    }

    struct mskwajd_header *hdr = kwajd->open(kwajd, input_file);
    if (!hdr) {
        mspack_destroy_kwaj_decompressor(kwajd);
        return 0;
    }

    if (hdr->filename != NULL)
        printf("Parsed filename: %s\n", hdr->filename);
    else
        printf("No filename parsed.\n");

    int ret = kwajd->extract(kwajd, hdr, "/dev/null");
    if (ret != MSPACK_ERR_OK) {
        fprintf(stderr, "Extraction failed with error code %d\n", ret);
    }

    kwajd->close(kwajd, hdr);
    mspack_destroy_kwaj_decompressor(kwajd);
    return 0;
}
```

Listing 2: KWAJ Fuzzing Harness for libmspack

A.2 libusbmuxd Harness

```c
#include <plist/plist.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>
int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
    if (size < sizeof(struct usbmuxd_header))
        return 0;

    struct usbmuxd_header hdr;
    memcpy(&hdr, data, sizeof(hdr));

    // Validate header length
    if (hdr.length > size || hdr.length < sizeof(hdr))
        return 0;

    // Treat the rest as binary plist
    plist_t payload = NULL;
    plist_from_bin((const char *)(data + sizeof(hdr)), size - sizeof(hdr), &
        ↪ payload);

    if (payload) {
        plist_free(payload);
    }

    return 0;
}
```

Listing 3: Fuzzing Harness for libusbmuxd

References

Ayala, J., Tung, Y., and Garcia, J. (2025). Investigating vulnerability disclosures in open-source software using bug bounty reports and security advisories. arXiv.

Nourry, O., Kondo, M., Saito, S., Iimura, Y., Ubayashi, N., and Kamei, Y. (2024). Myth: The loss of core developers is a critical issue for oss communities. arXiv.

Open Source Security Foundation (2022). Openssf best practices badge.

OWASP Foundation (2023). Top 10 risks for open source software.

Snyk (2023). State of open source security report.

Sonatype (2022). Sonatype finds 700% average increase in open source supply chain attacks. Accessed: 2025-04-27.

Synopsys (2023). Open source security and risk analysis report.

W3Techs (2020). Usage statistics and market share of linux for websites. Accessed: 2024-04-26.

Wermke, D., Wöhler, N., Klemmer, J. H., Fourné, M., Acar, Y., and Fahl, S. (2022). Committed to trust: A qualitative study on security & trust in open source software projects. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 1880–1896.

Zahan, N., Kanakiya, P., Hambleton, B., Shohan, S., and Williams, L. (2023). Openssf scorecard: On the path toward ecosystem-wide automated security metrics. *IEEE Security & Privacy*, 21(6):76–88.